Memory Leak Detection Runtime-Service for Embedded Linux Devices

Roman Beneder, Bernd Glatz, Martin Horauer
University of Applied Sciences Technikum Wien
beneder, glatz, horauer@technikum-wien.at
Austria, Vienna

Thomas Rauscher
LOYTEC electronics GmbH
trauscher@loytec.com
Austria, Vienna

***Abstract*—In practice, software is often deployed with several hidden bugs despite various test processes and static analyses. Such bugs often cause a phenomenon called software aging that refers to the accumulation of errors occurring in long running software systems that results in a decrease of performance and an increases of the probability to crash the entire system. This paper presents a mechanism to detect memory leaks in embedded systems software at runtime that can be used to counter software aging. In particular, it presents and compares two measurement based algorithms to identify memory leaks on Embedded Linux devices at runtime. Both algorithms have been implemented and evaluated using an industrial room controller targeting building automation. Based on this approach we were able to identify an existing memory leak we were unaware of beforehand.**

I. Introduction & Motivation

Software aging refers to the accumulation of errors occurring in long running software systems that typically results in a performance decrease and an increase of the probability leading to a system crash. Typical examples are dangling descriptors or process IDs, an increase of memory consumption due to memory leaks, memory fragmentation, etc. Once, software is deployed to an embedded target, identification of such aging-related effects is difficult and requires dedicated monitoring activities. Note, the 'standard way' here is to avoid respective effects beforehand using elaborate test, debug, and memory inspection tools when developing or verifying embedded software, respectively, cf. [1]. Nevertheless, such defects often find their way unnoticed to the target systems.

According to former studies (e.g. [2], [3], [4], [5]) a commonly used approach to detect software aging effects is based on the following general two steps:

(i) Selected system metrics (e.g., free or used memory), which are assumed to capture aging effects, are monitored under controlled workload conditions.

(ii) Based on the collected data set and supporting assumptions (e.g., aging is present if memory usage increases monotonically under constant workload), a statistical technique is used to detect trends that are considered as a sign of software aging.

A common way to deal with software aging is software rejuvenation [6]. Software rejuvenation is a preventive maintenance technique which periodically performs a software restart to bring the application or system to a clean internal state. Unfortunately, this technique results in (short) downtimes that are often not acceptable depending on the field of application. An alternative countermeasure to software aging is software life-extension [5]. This preventive maintenance technique prolongs the lifetime of software execution for as long as possible in face of aging. Due to the fact that this technique requires extensive amounts of memory, it is often not suitable for embedded applications.

In this paper, we introduce a hybrid technique to counter software aging caused by memory leaks. In particular, we present and evaluate two algorithms to detect memory leaks at runtime before we trigger countermeasures. This approach requires a minimum of additional memory and computing resources, which makes it suitable for embedded applications.

The following sections of this paper are structured as follows. Section II describes different techniques and methods in software aging and rejuvenation, respectively. Section III presents two different approaches (memory slope test and Mann-Kendall trend test) to detect memory leaks at runtime on an Embedded Linux device. Section IV highlights the results of the mentioned methods and compares them. Finally, section V surveys the literature and in section VI we state our conclusion.

II. Memory Leak Detection for Embedded Linux

There are three different techniques and methods to detect accumulation of errors in long running software systems.

1. Model-based analysis of data
2. Measurement-based analysis of data
3. Hybrid analysis of data

A. Model-Based Analysis

Model-based analysis relies on a mathematical model of the system with different states. It is mandatory to have at least a state where the system is working correctly, a state exhibiting software aging effects, and a state in which the rejuvenation technique will be processed. The main purpose of this approach is to increase the availability by identifying the optimal time for scheduling rejuvenation strategies.

Respective strategies are focusing on how a failure-prone state of a system is brought to an aging-free state. The goal is to keep the rejuvenating costs as low as possible, while a high availability and a reduction on performance loss should

2014 IEEE Emerging Technology and Factory Automation (ETFA)
978-1-4799-4845-1/14/$31.00 ©2014 IEEE
Authorized licensed use limited to: VIT University. Downloaded on August 05,2026 at 13:45:33 UTC from IEEE Xplore. Restrictions apply.

be achieved. The rejuvenation techniques can be separated into two different areas:

(a) Application-generic rejuvenation techniques rely on restarting device components or the complete system to bring it into the initial state, which is referred to be aging-free.

(b) Application-specific rejuvenation techniques are tailored to the system to reduce costs during the rejuvenation process by exploiting implementation specific knowledge.

The mathematical models used in model-based analysis are based on Markov chains and are a more general abstraction of a system. Thus, the portability to other systems is simpler compared to measurement-based approaches. Due to the generality of the model this approach of software rejuvenation is less effective compared to measurement-based approaches, because the measurement-based approach processes real device parameters.

B. Measurement-Based Analysis

This approach is based on gathering relevant data by directly observing system parameters (e.g., system temperature, file system or memory space) from a real system. The monitored parameters are named *aging indicators* and are used for predicting aging failures based on statistical data during runtime, cf. [7]. A common approach is analyzing the data based on its past observed behavior by performing a trend test followed by estimating such trends. Trend tests such as t-Student, Seasonal Kendall, or Mann-Kendall are checking if a trend is apparent in the observed statistical data.

C. Hybrid Analysis

Hybrid analysis is a combination of a model-based description of the system with direct measurements of the system parameters and additional workload information. This approach tries to gain advantage of both previously mentioned approaches. An implementation based on hybrid analysis is, e.g., detailed in [8] where a measurement based semi-Markov model was used parallel to observing the system workload.

III. Measurement-based Memory Leak Detection

This section focuses on measurement-based analysis for memory leak detection for Embedded Linux devices. While model-based analysis provides a higher portability, a measurement-based analysis approach better fits the needs of an embedded system where resources (e.g., performance, memory, etc.) are often limited.

The memory allocation data sets which are used for the analysis in this paper originate from an embedded device running an Embedded Linux derivate. The actual data was retrieved by utilizing the *mallinfo()* function to calculate the allocated memory.

A. Memory Leak Detection Based On Memory Slope

The first approach for memory leak detection in embedded systems presented in this paper relies explores the slope of the memory consumption while monitoring the number of processes. To be able to implement this approach it is necessary to record the allocated memory of the application and the corresponding timestamp. In order to be less sensitive to short memory peaks it is mandatory to take several independent data pairs (allocated memory, timestamp). This approach records n samples and calculates the memory slope between each data pair as shown in equation 1. The memory slope is calculated from the quotient of the difference of the sampled memory amount $Memory_n$ & $Memory_{n-1}$, and the difference of $Time_n$ & $Time_{n-1}$, while $Time_n$ represents a timestamp for each sample.

$$k_n = \frac{Memory_n - Memory_{n-1}}{Time_n - Time_{n-1}} = \frac{dMemory}{dTime} \qquad (1)$$

In order to determine the orientation of the overall memory slope, it is necessary to calculate the mean value of all memory slopes from equation 1.

Figure 1 shows a recorded sequence of memory slopes of an Embedded Linux device along with the mean value of the memory slope samples. Due to the fact that the tested application had no memory leaks, the overall memory slope is zero.

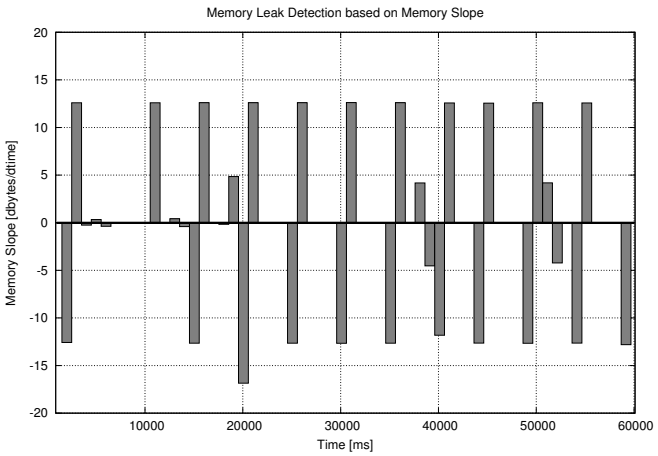


Fig. 1: Memory Slope

To be able to test the memory leak detection algorithm based on memory slopes a memory leak is applied periodically to the application. Within the application a function is called which allocates memory without freeing it. This memory allocation of the Embedded Linux device including the memory leak is shown in figure 2.

The height of every memory slope in figure 2 indicates the allocation at the recording time. A value near zero indicates that the application works correctly and a high positive value represents a memory leak. Furthermore, the mean value of all memory slopes in this recorded sequence is greater than zero

Authorized licensed use limited to: VIT University. Downloaded on August 05,2026 at 13:45:33 UTC from IEEE Xplore. Restrictions apply.

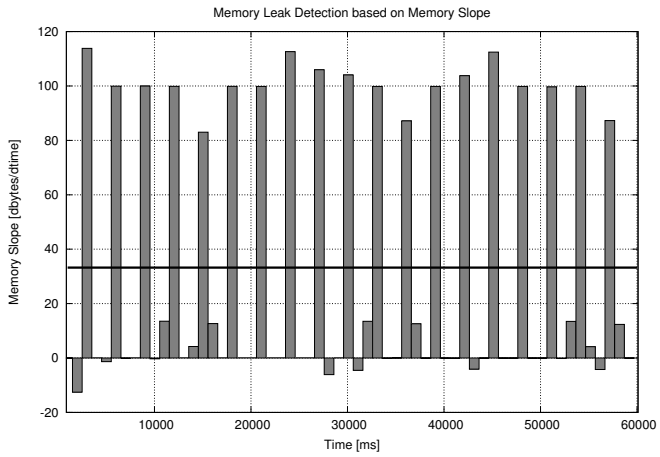


Fig. 2: Memory Slope with Memory Leak

(for the givenm example it is 33.144). This indicates that at least one application exhibits a memory leak.

B. Memory Leak Detection Based On Mann-Kendall

As explained in [2] the Mann-Kendall test [9], [10] verifies the null hypothesis, H_0, indicating that there is no trend over time, against the alternative hypothesis, H_1, presenting there is a upward or a downward monotonic trend against zero slope. Let $Y = (t_i, y_i)$ be a time series data, where y_i is the series value at time point t_i ($i \in (0...M)$). Given n consecutive data points, the Mann-Kendall statistic S (Kendall Score) is computed by

$$S = \sum_{k=1}^{n-1} \sum_{l=k+1}^{n} sgn(y_l - y_k) \tag{2}$$

where sgn denotes the signum function

$$sgn(x) = \begin{cases} -1, if\ x < 0 \\ 0, if\ x = 0 \\ 1, if\ x > 0 \end{cases} \tag{3}$$

The value of the Kendall Score computed via equation 2 is compared against a critical value. Based on this comparison the appropriate countermeasures can be initiated. The upper and the lower level of the critical value can be calculated with equation 4. For example, the maximum Kendall Score of a data series with 60 samples is 1770.

$$S_{max} = \sum_{i=1}^{n-1} i \tag{4}$$

A good approach for determining the critical value is to take 80% of the S_{max} value and for the log value, where the application triggers log messages, 60% of the S_{max} value.

To verify that the Mann-Kendall algorithm is suitable for an embedded system we applied the Mann-Kendall algorithm to a time series of samples of the actual allocated heap. If the Kendall Score is a significant positive value, a rising trend is detected and indicates that a memory leak is apparent. If the Kendall Score is nearly zero neither a rising nor a falling trend is detected and it is implied that the application does not have a memory leak. A significant negative value implies a falling trend.

Figure 3 shows a time series of samples of the memory allocation history of an Embedded Linux application and the corresponding Kendall Score S. The recorded data set (data set A in table I) shows that this application allocates memory and frees only parts of it again.

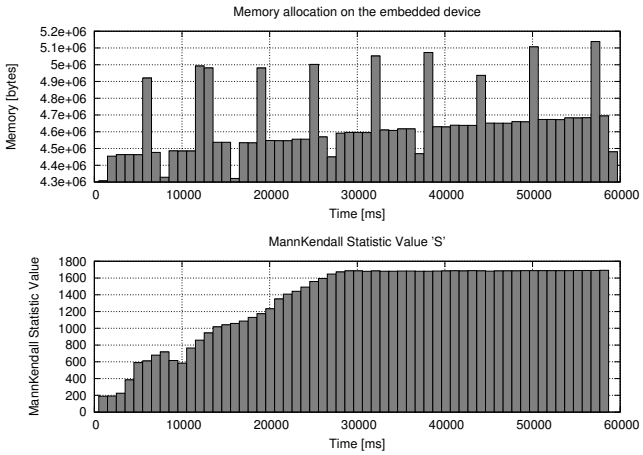


Fig. 3: Memory Leak on an Embedded Linux Device

Figure 4 likewise shows the memory allocation of the same Embedded Linux application which was used to generate the data set (data set B in table I) for figure 3.

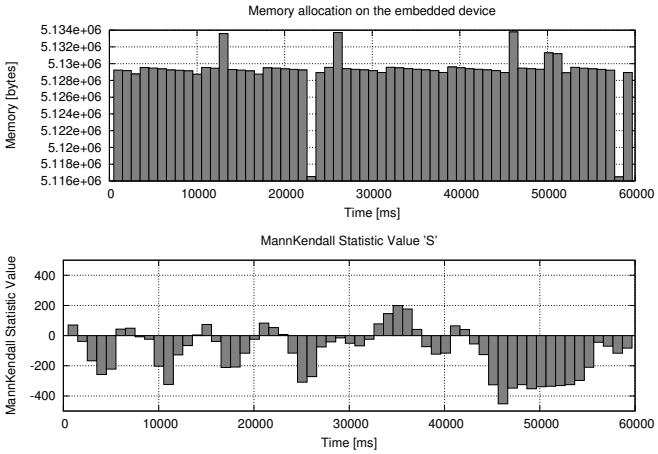


Fig. 4: Mann Kendall No Memory Leak

To be able to calculate a meaningful Kendall Score it is necessary – according to [10] – to take more than 10 samples for the data set which will be analyzed by the Mann-Kendall algorithm. Additionally figure 3 shows the Kendall Score which indicates an upwards trend in case a memory leak is detected on the Embedded Linux device. The Mann-Kendall trend analysis provided the parameters (shown in table I) for the data set shown in figure 3.

The recorded data set shows that the fixed application handles the memory allocation properly. Table I shows the Mann-

Authorized licensed use limited to: VIT University. Downloaded on August 05,2026 at 13:45:33 UTC from IEEE Xplore. Restrictions apply.

Kendall components resulting from data set A (application with memory leak) and data set B (application without memory leak). The comparison of the Kendall Score of each data set proves that the application based on data set A with the Kendall Score S = 990 has a memory leak. The Kendall Score of data

Mann-Kendall components	Results for data set A	Results for data set B
τ	0.561	-0.133
sl	0	0.139
S	990	-227
D	1765	1704
var(S)	24573	23369

TABLE I: Mann-Kendall component comparison

set B (S = -227) proves that the memory leak of the application was fixed.

In order to examine the behavior of the Mann-Kendall algorithm in more detail a memory leak with different slopes was applied to the application under test. Figure 5 shows the memory allocation of the application in case a memory slope of 33.3 kByte/s is applied. Additionally figure 5 shows the Kendall Score which indicates an upwards trend in case a memory leak is detected on the Embedded Linux device. The value of the Kendall Score is positive which suggests that an upwards trend has been detected. The mean value of the Kendall Score per group of sample with the applied memory leak is $\overline{S}$ = 33.17.

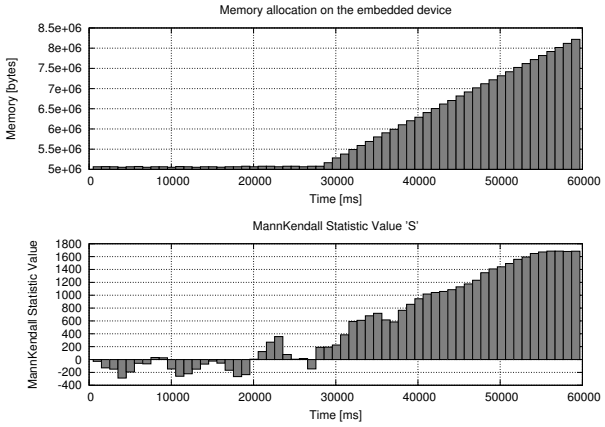


Fig. 5: Mann Kendall Progressive Memory Leak 33.3 kByte/s

Figure 6 shows the memory allocation of the application in case a memory slope of 333.3 kByte/s is applied. The value of the Kendall Score is positive which suggests that an upwards trend has been detected. The mean value of the Kendall Score per group of sample with the applied memory leak is $\overline{S}$ = 35.17.

Table II highlights the corresponding Mann-Kendall components of the time series of samples with two memory slopes. Based on the comparison of the Kendall Score values of both data sets the actual amount of non freed memory per second does not effect the result of the Mann-Kendall algorithm.

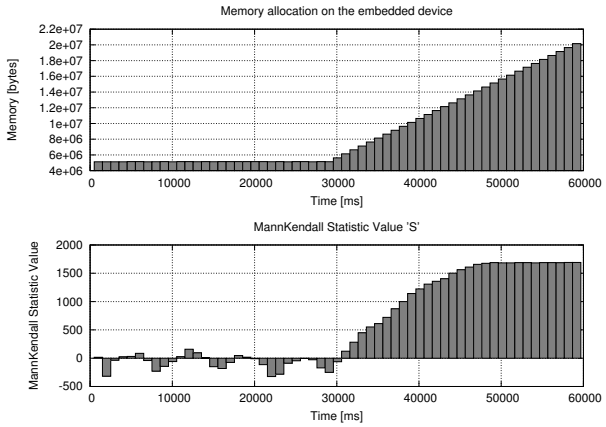


Fig. 6: Mann Kendall Progressive Memory Leak 166 kByte/s

Mann-Kendall components	Memory Slope k = 33.3 kBytes/s	Memory Slope k = 333.3 kBytes/s
τ	0.896	0.96
sl	0	0
S	1533	1585
D	1710	1711
var(S)	23381	23384
$\overline{S}$	33.17	35.17

TABLE II: Mann-Kendall component comparison of two different memory slopes

This means that it is possible to detect a memory leak of an embedded application. The Mann-Kendall algorithm itself, however, does not provide a statement regarding the actual amount of non freed memory.

IV. Comparison Of The Memory Leak Detection Algorithms

In III-A and III-B memory leak detection algorithms were explained. This section highlights a comparison of the algorithms regarding memory efficiency and time to detect the memory leak shown in table III.

Memory Leak Detection	Memory Slope	Mann-Kendall
Elements in time series	60	60
Heap [Bytes]	2928	1952
Detection time (33.3 kB/s) [s]	18.7	36.14
Detection time (3.3 kB/s) [s]	-	36.14

TABLE III: Memory Leak Detection Algorithms Comparison

The measurement-based memory leak detection algorithm based on memory slope retrieves the information to calculate the memory slope out of a set of 60 samples. In case the time difference between each sample is constant, there is no need to take a timestamp for each sample. In order to calculate a memory slope for a time series of samples, where the time differential between each sample can vary, it is mandatory to save the according timestamps for each sample. The advantage

Authorized licensed use limited to: VIT University. Downloaded on August 05,2026 at 13:45:33 UTC from IEEE Xplore. Restrictions apply.

of utilizing the Mann-Kendall algorithm for the memory leak detection is, that regardless of the time differential between the samples, it is possible to calculate a monotonic trend. To be able to provide the most versatile usage of each proposed algorithm, the memory slope algorithm III-A stores additional timestamps for each sample whereas the Mann-Kendall algorithm does not need time related information.

According to table III the memory slope algorithm itself requires 2928 bytes of allocated memory to store the time series. Due to the fact that the memory leak detection algorithm based on Mann-Kendall assumes that the data set is already a time series, the Mann-Kendall algorithm does not need timestamps in its data set. Therefore, the Mann-Kendall algorithm requires less information and only needs 1952 bytes of allocated memory. This causes a memory reduction of 33% by using the Mann-Kendall algorithm instead of the memory slope calculation.

To be able to examine which of the introduced memory leak detection algorithms is faster a memory leak of 33.3 kBytes/s was applied to the application and the time until the start of the memory leak and the detection of the memory leak was recorded. In case the memory leak detection based on memory slope is used, it takes 18.7 seconds to detect a memory leak. The Mann-Kendall algorithm with a detection time of 36.14 seconds. This means for a memory slope of 33.3 kBytes/s the memory slope algorithm is twice as fast as the Mann-Kendall algorithm. However, it also means that the memory slope algorithm is heavily dependent on actual height of the memory slope. The detection time of the memory slope algorithm decreases in case the memory leak increases. Additionally it has a lower bound limit where no leaks are detectable.

For example the critical value for the memory slope in this application is set to 10; this means that memory leaks of 10 kBytes/s or more are detectable. On the other hand, the Mann-Kendall algorithm does not depend on the height of the memory leak. It is an algorithm to detect trends and therefore a small memory leak of 3.3 kBytes/s requires, according to III, the same time to detect the memory leak as a memory leak of 33 kBytes/s or any other memory leak. The Mann-Kendall algorithm requires fewer resources when compared to the memory slope algorithm, thus it is better suited for Embedded Linux applications.

V. Related Work

In [2] an approach to test the effectiveness of the Mann-Kendall trend detection test concerning software aging was introduced. The experiment used in this paper focused on detecting software aging for a Linux system application. This approach uses the free system memory, which is a system-wide aging indicator, and various application specific indicators (e.g. *Resident Set Size*, etc.). The experiment uncovered that the choice of the underlying system metrics as aging indicators have a significant impact on the aging detection capability. For example, the free memory metric tends to be useless whereas the *Resident Set Size* and the *Heap Usage* turned out to be useful. Based on the uncovered usefulness of the Mann-Kendall trend detection test for detecting memory leaks, the approach described in this paper, monitors the *Heap Usage* and initiates countermeasures for trend violations.

In [3] a methodology for detecting and estimating software aging was introduced. Based on a Simple Network Management Protocol (SNMP) connection between a workstation and an embedded target a distributed resource monitoring and data collection tool was developed to gather system dependent information. The retrieved information is stored on the workstation and within the monitoring tool it is possible to apply a seasonal Mann-Kendall trend detection test. Furthermore this paper proposed a metric, "estimated time to exhaustion" for each resource as a quantification of aging. The higher this metric is, the lower is the effect of this resource. This metric helps in comparing the effect of aging on different system resources and also in the identification of important resources to monitor and manage.

The study reported in [4] focuses on the analysis of software aging in the Linux kernel. The experimental procedure revealed that the Linux OS suffers from software aging phenomena, manifested as statistically significant memory consumption trends. Furthermore, [4] points out that memory consumption is a more severe issue than the performance loss in embedded devices and therefore the Mann-Kendall trend detection test was implemented. They claim that the Mann-Kendall test detects memory leaks with at least 95% confidence. The results discovered in this paper can be an aid for Linux kernel developers to detect software aging bugs in large software applications.

The countermeasure of software aging presented in [5] claims to extend the lifetime of software execution before encountering a failure caused by resource exhaustion. The primary advantage of the presented concept is that software life extension (compared to software rejuvenation) doesn't mandate a restart. In some types of software, the accumulated data in memory is important for its services and performance and software life extension attempts to preserve the memory content as long as possible. This concept was designed for high available IT systems and not for embedded devices where memory is a costly resource. The amount of free memory and swap usage are changed by software-life extension thus the virtual machine does not immediately go out of memory. The swap usage fluctuates around a certain amount of MBytes with a subtle upward trend. This upward trend was confirmed by the Mann-Kendall trend test.

VI. Conclusion

Software aging presents a severe threat in todays' business- and high-availabile software systems. The progress of software aging can be monitored based on various system metrics. A typical example of software aging is due to a progressive increase in memory consumption caused by memory leaks. The most commonly used countermeasure is software rejuvenation, where system restarts are performed periodically. Due to the fact that these system restarts decrease the availability it is mandatory to provide runtime techniques to detect system metric violations.

This paper introduces two approaches to detect memory leaks on embedded devices. The detection technique described in subsection III-A is based on calculating memory slopes out of a recorded set of data. The disadvantage of this approach is that a minimum amount of memory leakage has to be present,

Authorized licensed use limited to: VIT University. Downloaded on August 05,2026 at 13:45:33 UTC from IEEE Xplore. Restrictions apply.

to be able to detect the memory leak. Due to the fact that the usage of this technique does not identify every kind of memory leak, a statistical analysis to detect trends of time series based data sets is described in subsection III-B.

In order to detect trends it is possible to utilize various trend detection algorithms. Trend tests such as Mann-Kendall, Seasonal Kendall or t-Student are supposed to check whether there is a trend in the given data set. The Mann-Kendall test requires a data set over a time series and therefore the test does not need timestamps. This leads to a smaller memory footprint which is beneficial for embedded devices. Moreover the Mann-Kendall trend test provides the Kendall Score which is an informative value about the direction of the trend. In case the Kendall Score is significantly positive, it means that an upwards trend has been detected. This means that the amount of used memory increases under normal operation, which implies that the application has a memory leak. In case the Kendall Score is significant negative, a downwards trend is detected and if the Kendall Score is nearly zero, the mean value of the applications memory usage is constant. Both, the horizontal and the negative trend, are neither critical values for embedded systems. The main advantage of the Mann-Kendall algorithm is that all trends are detectable, even if the memory leak is very small.

Utilizing the proposed methodology it was possible to identify a memory leak on an Embedded Linux device which is supposed to execute long running software.

ACKNOWLEDGMENT

This work has been supported by LOYTEC electronics GmbH and by the public funded R&D project Josef Ressel Center for Verification of Embedded Computing Systems (VECS) managed by the CDG.

REFERENCES

[1] G. Chatzieleftheriou, P. Katsaros, *Test-driving static analysis tools in search of C code vulnerabilities*, The 35st Annual Computer Software and Applications Conference Workshops (COMPSACW), 18-22 July 2011, Munich, Germany.

[2] F. Machida, A. Andrzejak, R. Matias, E. Vicente, *On the Effectiveness of Mann-Kendall Test for Detection of Software Aging*, WoSAR 2013, The 5th International Workshop on Software Aging and Rejuvenation, Nov 4-7, Pasadena, CA, USA.

[3] S. Garg, Aad van Moorsel, K. Vaidyanathan, K.S. Trivedi, *A Methodology for Detection and Estimation of Software Aging*, The 9th International Symposium on Software Reliability Engineering, Nov 4-7 1998, Paderborn, Germany.

[4] D. Cotroneo, R. Natella, R. Pietrantuono, S. Russo, *Software Aging Analysis of the Linux Operating System*, The 21st International Symposium on Software Reliability Engineering, Nov 1-4 2010, San Jose, CA, USA.

[5] F. Machida, J. Xiang, K. Tadano, Y. Maeno , *Software Life-extension: A New Countermeasure to Software Aging*, The 23st International Symposium on Software Reliability Engineering, Nov 27-30 2012, Dallas, TX, USA.

[6] Y. Huang, C. Kintala, N. Kolettis, D. Dudley Fulton, *Software Rejuvenation: Analysis, Module and Applications*, The 25st International Symposium on Fault-Tolerant Computing, Dec 27-30 1995, Pasadena, CA, USA.

[7] D. Cotroneo, R. Natella, R. Pietrantuono, S. Russo, *A Survey of Software Aging and Rejuvenation Studies*, ACM Journal on Emerging Technologies in Computing Systems (JETC), Vol. 10, Issue 1, No. 8, 2014.

[8] Kalyanaraman Vaidyanathan, Kishor S. Trivedi, *A Comprehensive Model for Software Rejuvenation*, IEEE Transactions on Dependable and Secure Computing, Vol 2, Issue 2, pp. 124–137, 2005.

[9] H.B. Mann, *Nonparametric Tests Against Trend*, Econometrica, Vol. 13, No. 3, pp. 245-259, Jul. 1945.

[10] M.G. Kendall, *Rank Correlation Methods*, Charles Griffin & Company Limited, 4th Edition, 1970.

Authorized licensed use limited to: VIT University. Downloaded on August 05,2026 at 13:45:33 UTC from IEEE Xplore. Restrictions apply.